

8.4 外推算法与龙贝格算法

主讲：施明辉

厦门大学

The background of the slide features several sets of concentric circles in a lighter blue shade, resembling ripples on water, positioned in the lower right and bottom center areas.

8.4 外推算法与龙贝格算法

8.4.1 变步长的求积公式

8.4.2 外推算法

8.4.3 龙贝格求积公式

8.4.1 变步长的求积公式

在利用复合求积公式计算积分积分之前，必须给出适当的步长 $(h = \frac{b-a}{n})$ ，步长太大，精度难以保证，步长太小，则会导致计算量的增加，而计算之前给出一个恰当的步长，往往非常困难，因此，在实际计算中，常采用变步长的方法。

基本思想是：在步长逐次折半的过程中，反复用复合求积公式进行计算，直到步长折半前后的两次积分值之差的绝对值

$|I_{2n}(f) - I_n(f)|$ 小于允许的精度为止，并取 $I_{2n}(f)$ 作为所求积分的近似值。

以复合梯形求积公式为例，将区间 $[a, b]$ 分成 n 等分，步长 $h = \frac{b-a}{n}$ ，则：

$$T_n = T(h) = \frac{h}{2} \left[f(a) + 2 \sum_{i=1}^{n-1} f(x_i) + f(b) \right] \quad (8.4.1)$$

若将区间 $[a,b]$ 分为 $2n$ 等分，步长为原来的一半，变为 $\frac{h}{2} = \frac{b-a}{2n}$ ，则：

$$\begin{aligned} T_{2n} &= T\left(\frac{h}{2}\right) = \frac{1}{2}\left(\frac{h}{2}\right) \left[f(a) + 2\sum_{i=1}^{n-1} f(x_i) + 2\sum_{i=0}^{n-1} f\left(x_{i+\frac{1}{2}}\right) + f(b) \right] \\ &= \frac{1}{2}T_n + \frac{h}{2}\sum_{i=0}^{n-1} f\left(x_{i+\frac{1}{2}}\right) \end{aligned} \quad (8.4.2)$$

由于复合梯形公式是收敛的，所以用逐次折半的方法构造出的近似值系列，收敛于精确值。

$$T(h), T\left(\frac{h}{2}\right), T\left(\frac{h}{2^2}\right), \dots, T\left(\frac{h}{2^n}\right), \dots$$

例8.4.1 用变步长的梯形法求积分 $I(f) = \int_0^1 \frac{\sin x}{x} dx$ 的近似值。

解：补充定义 $f(0)=1$ ，计算 $f(1)=0.8414709$

所以 $T_1 = \frac{1}{2}[f(0) + f(1)] = 0.920735$

将 $[0,1]$ 二等分，计算 $f(\frac{1}{2}) = 0.9588510$

利用 (8.4.2) 得：

$$T_2 = \frac{1}{2}T_1 + \frac{1}{2}f(\frac{1}{2}) = 0.9397933$$

将 $[0,1]$ 四等分，计算 $f(\frac{1}{4}) = 0.9896158, f(\frac{3}{4}) = 0.9088516$

利用 (8.4.2) 得：

$$T_4 = \frac{1}{2}T_2 + \frac{1}{4}\left[f\left(\frac{1}{4}\right) + f\left(\frac{3}{4}\right)\right] = 0.9445135$$

这样，将区间逐次二分，步长逐次折半，计算结果见表8.4.1（表中， k 为二分次数，区间分数为 $n = 2^k$ ）

表8.4.1 步长逐次折半的计算结果

K	0	1	2	3	4	5
T_{2^k}	0.9207355	0.9397933	0.9445135	0.9456909	0.9459850	0.9460586
K	6	7	8	9	10	11
T_{2^k}	0.9460769	0.9460815	0.9460827	0.9460830	0.9460830	0.9460831

积分 $I(f)$ 的精确值为0.9460831，用变步长的复合梯形公式步长逐次折半11次以后，才得到了这个结果，这说明，收敛速度非常缓慢。



8.4.2 外推算法

在数值计算中，常常利用一个序列 $T_1, T_2, \dots, T_n, \dots$ 去逼近准确解 T ，序列 $\{T_i\}, (i=1, 2, \dots)$ 又经常与区间的步长有关。例如用变步长的复合梯形公式产生一个序列： $T(h), T(\frac{h}{2}), \dots, T(\frac{h}{2^n}), \dots$ ，去逼近准确值 $T(0)$ 。由于这一序列，收敛的速度很慢，如何提高收敛速度以节省计算量便成了一个最实际的问题。

这一问题，可以推广到更一般的情况：

给定一个收敛于 $f(0)$ 的序列 $f(h), f(\frac{h}{2}), \dots$ ，能否在此基础上构造一个新的序列，使其更快的收敛于 $f(0)$ 。这在某些条件下是可以实现的，例如利用泰勒(Taylor)展开式：

$$f(h) = f(0) + hf'(0) + \frac{h^2}{2!} f''(0) + \frac{h^3}{3!} f'''(0) + \dots \quad (8.4.3)$$

$$f\left(\frac{h}{2}\right) = f(0) + \frac{h}{2} f'(0) + \frac{1}{2!} \left(\frac{h}{2}\right)^2 f''(0) + \frac{1}{3!} \left(\frac{h}{2}\right)^3 f'''(0) + \dots \quad (8.4.4)$$

如果 $f'(0) \neq 0$ 那么 $f(h), f(\frac{h}{2})$ 逼近 $f(0)$ 的阶都是 $O(h)$

现将 (8.4.4) 乘以 2，减去式 (8.4.3) 得

$$f_1(h) = 2f\left(\frac{h}{2}\right) - f(h) = f(0) - \frac{h^2}{4} f''(0) - \frac{h^3}{8} f'''(0) + \dots$$

如果 $f''(0) \neq 0$, 那么 $2f(\frac{h}{2}) - f(h)$ 逼近 $f(0)$ 的误差阶为 $O(h^2)$ 。

即新序列 $f_1(h) = 2f(\frac{h}{2}) - f(h)$ 有可能比旧序列 $f(h)$ 更快地收敛于 $f(0)$ 。

可以继续从序列 $f_1(h)$ 再产生出新的序列 $f_2(h)$, 使其更快的收敛于 $f(0)$, 由于:

$$f_1(h) = f(0) - \frac{h^2}{4} f''(0) - \frac{h^3}{8} f'''(0) + \dots$$

$$f_1\left(\frac{h}{2}\right) = f(0) - \frac{h^2}{16} f''(0) - \frac{h^3}{64} f'''(0) + \dots$$

后式乘以4，减去前一式，有：

$$h_2(f) = \frac{4f_1\left(\frac{h}{2}\right) - f_1(h)}{3} = f(0) + \frac{h^3}{48}f'''(0) + \dots$$

如果 $f'''(0) \neq 0$ ，则序列 $\{f_2(h)\}$ 逼近 $f(0)$ 的误差阶为 $O(h^3)$

上述推理过程，是利用若干近似值推算法更精确的近似值的加速收敛的方法，这种方法称为外推算法，外推算法在数值积分和数值微分等数值计算中的都有广泛的应用。



8.4.3 龙贝格求积公式

用变步长的求积公式，可以根据精度的要求，在计算过程中适当调整步长，使其计算结果逐步逼近精确值，但是近似值序列收敛于积分精确值的速度较慢，现在利用外推算法的原理，来研究提高收敛速度的方法。

递推关系

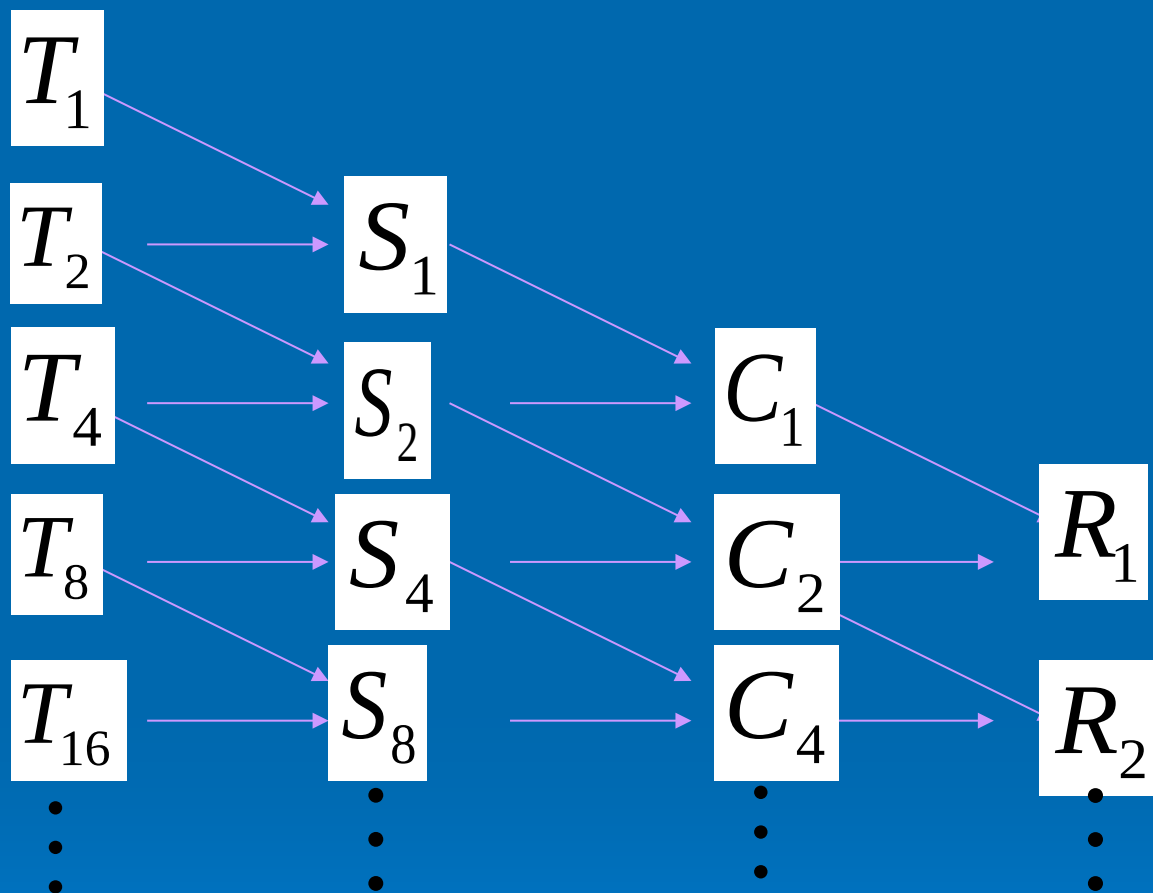
$$S_n = \frac{4}{4-1}T_{2n} - \frac{1}{4-1}T_n$$

$$S_1 = \frac{4}{3}T_2 - \frac{1}{3}T_1 = \frac{4}{4-1}T_2 - \frac{1}{4-1}T_1$$

$$S_2 = \frac{4}{4-1}T_4 - \frac{1}{4-1}T_2$$

$$C_n = \frac{4^2}{4^2-1}S_{2n} - \frac{4}{4^2-1}S_n$$

$$R_n = \frac{4^3}{4^3-1}C_{2n} - \frac{1}{4^3-1}C_n$$



(图8.4.1 龙贝格算法流程)

龙贝格求积公式的推导

由复合梯形法的误差公式 (8.3.3) 式知,
积分值 T_n 的截断误差为:

$$I - T_n = -\frac{b-a}{12} h^2 f''(\eta_1)$$

它与 h^2 成正比, 若把区间再对分, 即 $2n$ 等分,
则截断误差为:

$$I - T_{2n} = -\frac{b-a}{12} \left(\frac{h}{2}\right)^2 f''(\eta_2)$$

如果 $f''(x)$ 在 $[a, b]$ 上变化不大, 即有:

$$f''(\eta_1) \approx f''(\eta_2)$$

则由上述两个误差可得:

$$\frac{I - T_n}{I - T_{2n}} \approx 4$$

从而, $I \approx \frac{4}{3}T_{2n} - \frac{1}{3}T_n = \bar{T}$

计算结果 $\bar{T}$ 应该比 T_n 和 T_{2n} 更精确。

当 $n=1$ 时, 直接计算有:

$$\bar{T} = \frac{4}{3}T_2 - \frac{1}{3}T_1 = \frac{4}{3} \times \frac{1}{2} \left(\frac{b-a}{2} \right) [f(a) + 2f\left(\frac{a+b}{2}\right) + f(b)]$$

$$- \frac{1}{3} \times \frac{b-a}{2} [f(a) - f(b)]$$

$$= \frac{b-a}{6} [f(a) + 4f\left(\frac{a+b}{2}\right) + f(b)]$$

这正是 $[a, b]$ 上应用辛普生求积公式的结果, 即

$$S_1 = \frac{4}{3}T_2 - \frac{1}{3}T_1 = \frac{4}{4-1}T_2 - \frac{1}{4-1}T_1$$

可以验证：

$$S_2 = \frac{4}{4-1}T_4 - \frac{1}{4-1}T_2$$

恰好是将 $[a,b]$ 二等分后，在每个小区间上应用辛普生求积公式的结果，一般地有：

$$S_n = \frac{4}{4-1}T_{2n} - \frac{1}{4-1}T_n \quad (8.4.5)$$

S_n 是区间 $[a,b]$ n 等分后，在每个小区间上应用抛物线求积公式的结果，这说明，用复合梯形法

二分前后的两个积分近似值 T_n 和 T_{2n} 按 (8.4.5) 式计算, 就可以得到精度较高的辛普生求积公式的近似值 S_n , 从而加速了逼近的效果, 所以称 (8.4.5) 式为梯形加速公式。

由复合辛普生法的误差公式（8.4.5）式知，积分值 S_n 的截断误差为：

$$I - S_n = \frac{b-a}{180} \left(\frac{h}{2}\right)^4 f^{(4)}(\eta_1)$$

与 h^4 成比例，如果 $f^{(4)}(x)$ 在 $[a,b]$ 区间上变化不大，则将步长折半后，误差将减至原有误差的 $\frac{1}{16}$ ，即有：

$$\frac{I - S_{2n}}{I - S_n} \approx \frac{1}{16}$$

$$I \approx \frac{16}{15} S_{2n} - \frac{1}{15} S_n$$

$$I \approx \frac{16}{15} S_{2n} - \frac{1}{15} S_n$$

可以验证，上式右端的值,是将积分区间 $[a,b]$ 等分后，在每个小区间上用柯特斯公式得到的具有更高精度的复合柯特斯公式的积分值 C_n ，即有：

$$C_n = \frac{4^2}{4^2 - 1} S_{2n} - \frac{4}{4^2 - 1} S_n \quad (8.4.6)$$

(8.4.6) 式称为**辛普生加速公式**。

重复同样的方法，依据复合柯特斯公式的误差公式，可进一步得出公式：

$$R_n = \frac{4^3}{4^3 - 1} C_{2n} - \frac{1}{4^3 - 1} C_n \quad (8.4.7)$$

由(8.4.7)计算出的序列 $R_1, R_2, \dots, R_n, \dots$ 逼近积分精确值的速数又会迅速得到很大提高。

按照上述思路，还可以构造出新的求积公式，且右端有两个系数，分别为 $\frac{4^m}{4^m - 1}$ 及 $\frac{1}{4^m - 1}$ ，但当 $m \geq 4$ 时，第一个系数接近于1，第二个系数绝对值很小，因此，这样组合的新公式与前一个公式计算结果没有多大区别，反而增加了计算的工作量，所以，在实际计算时，只计算到(8.4.7)式为止。通常将变步长梯形法的积分的近似值利用(8.4.5)，(8.4.6)，(8.4.7)加速公式加工成精确度更高的积分近似值的方法称为**龙贝格**

(Romberg)求积算法。

特别地，(8.4.7)式为龙贝格求积公式。

由于计算 R_1 ，需要先计算 C_2, S_4, T_8 即将 $[a, b]$ 8等分，计算9个节点的函数值，但从公式的推导过程可知，的误差阶为 $O(h^7)$ ， R_1 代数精度为7次，因此，它已不属于插值型求积公式，也不属于牛顿-柯特斯求积公式的范畴。

用龙贝格方法计算积分的步骤为:

(1) : 准备初值, 先用梯形公式计算积分近

似值: $T_1 = \frac{b-a}{2} [f(a) + f(b)]$

(2) : 按变步长梯形公式计算积分近似值:

令 $h = \frac{b-a}{2^i} \quad (i = 0, 1, 2, \dots)$

计算:

$$T_{2n} = \frac{1}{2} T_n + \frac{h}{2} \sum_{i=0}^{n-1} f\left(x_{i+\frac{1}{2}}\right) \quad n = 2^i$$

(3) : 按加速公式求积分 (为便于编程, 写为下列形式)

梯形加速: $S_n = T_{2n} + (T_{2n} - T_n)/3$

辛普生加速: $C_n = S_{2n} + (S_{2n} - S_n)/15$

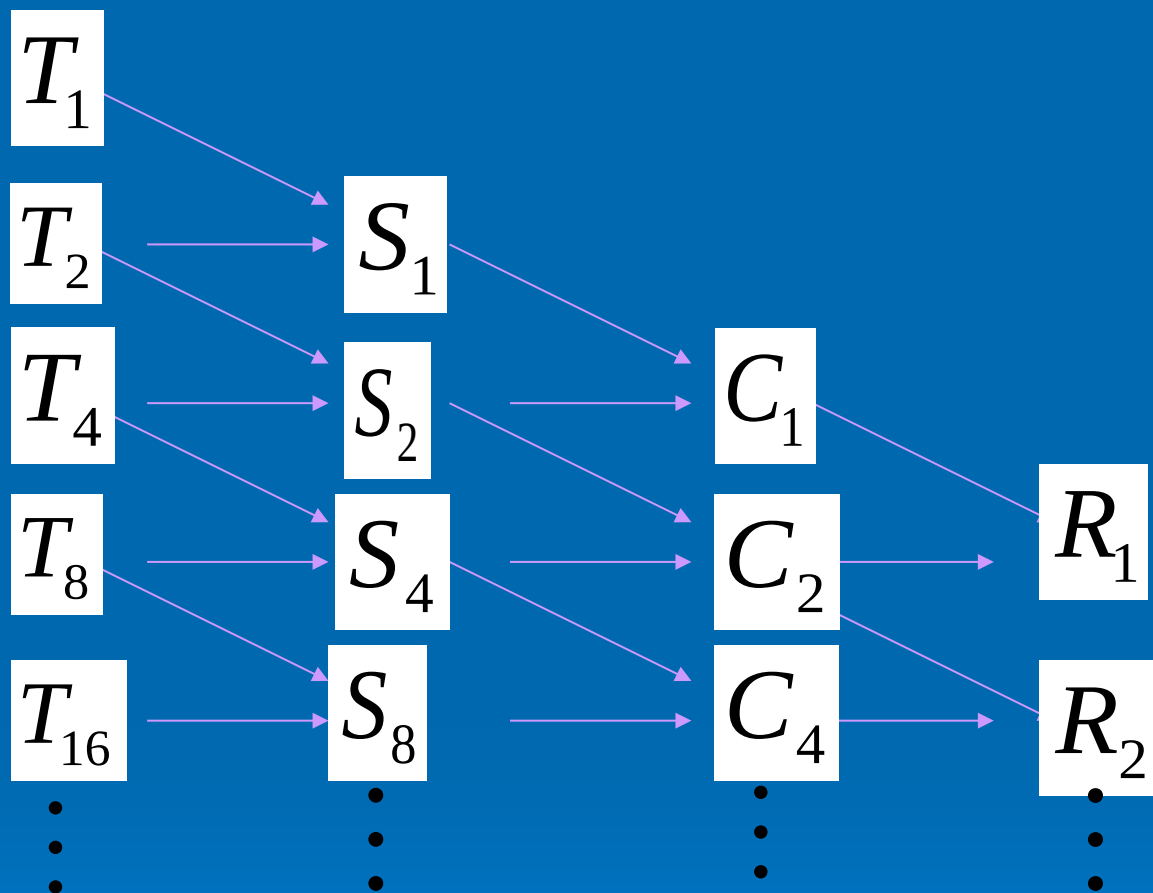
柯特斯加速: $R_n = C_{2n} + (C_{2n} - C_n)/63$

(4) : 精度控制

当 $|R_{2n} - R_n| < \varepsilon$, (ε 为精度) 时终止计算

并取 R_{2n} 为近似值, 否则, 将步长折半, 转

(2) 执行。实际计算时的加工流程如图8.4.1。



(图8.4.1 龙贝格算法流程)

例8.4.2 用龙贝格算法加工例8.4.1得到的近似值

解：用算法图8.4.2计算结果见表8.4.1,表中

k 表示二分次数：

k	T_{2^k}	$S_{2^{k-1}}$	$C_{2^{k-2}}$	$K_{2^{k-3}}$
0	0.9207355	0.9461459 0.9460869 0.9460834	0.9460830 0.9460831	0.9460831
1	0.9397933			
2	0.9445135			
3	0.9456909			

从表8.4.1的结果可以看出，用二分3次的精度很低的数据通过三次加速得到了例8.4.1需要二分10次才能求得的结果，而加速过程的计算量只需几次算术运算，而不需求函数值，算术运算可以忽略不计，由此可见，龙贝格加速过程的效果是非常明显的。

